

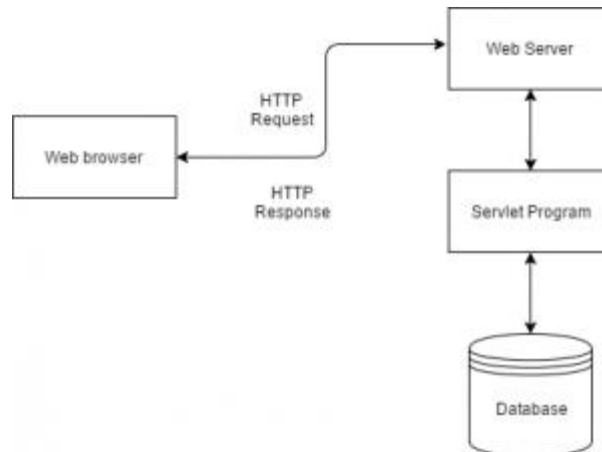
Introduction to Java Servlets

Servlets are the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the webserver, process the request, produce the response, then send a response back to the webserver.

Properties of Servlets are as follows:

- Servlets work on the server-side.
- Servlets are capable of handling complex requests obtained from the webserver.

Servlet Architecture is can be depicted from the image itself as provided below as follows:



Execution of Servlets basically involves six basic steps:

1. The clients send the request to the webserver.
2. The web server receives the request.
3. The web server passes the request to the corresponding servlet.
4. The servlet processes the request and generates the response in the form of output.
5. The servlet sends the response back to the webserver.
6. The web server sends the response back to the client and the client browser displays it on the screen.

Now let us do discuss eccentric point that why do we need For Server-Side extensions?

The **server-side extensions** are nothing but the technologies that are used to create dynamic Web pages. Actually, to provide the facility of dynamic Web pages, Web pages need a container or Web server. To meet this requirement, independent Web server providers offer some proprietary solutions in the form of **APIs**(Application Programming Interface).

These **APIs** allow us to build programs that can run with a Web server. In this case, **Java Servlet** is also one of the component APIs of **Java Platform Enterprise Edition** which sets standards for creating dynamic Web applications in Java.

The Servlet technology is similar to other Web server extensions such as **Common Gateway Interface**(CGI) scripts and **Hypertext Preprocessor** (PHP). However, Java Servlets are more acceptable since they solve the limitations of **CGI** such as low performance and low degree scalability.

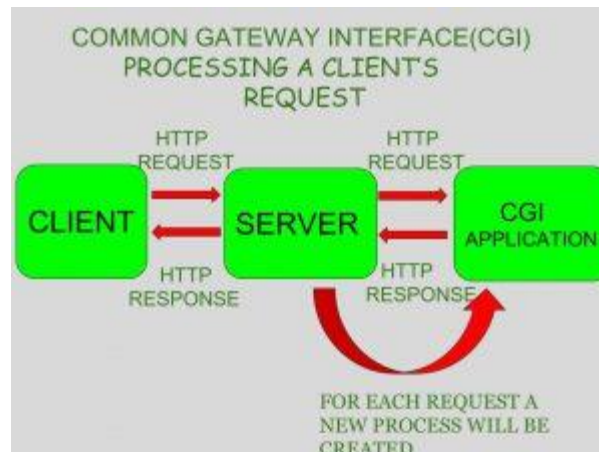
What is CGI?

CGI is actually an external application that is written by using any of the programming languages like **C** or **C++** and this is responsible for processing client requests and generating dynamic content.

In CGI application, when a client makes a request to access dynamic Web pages, the Web server performs the following operations :

- It first locates the requested web page *i.e* the required CGI application using URL.
- It then creates a new process to service the client's request.
- Invokes the CGI application within the process and passes the request information to the application.
- Collects the response from the CGI application.

- Destroys the process, prepares the HTTP response, and sends it to the client.



So, in **CGI** server has to create and destroy the process for every request. It's easy to understand that this approach is applicable for handling few clients but as the number of clients increases, the workload on the server increases and so the time is taken to process requests increases.

Difference between Servlet and CGI

Servlet	CGI(Common Gateway Interface)
Servlets are portable and efficient.	CGI is not portable
In Servlets, sharing data is possible.	In CGI, sharing data is not possible.
Servlets can directly communicate with the webserver.	CGI cannot directly communicate with the webserver.
Servlets are less expensive than CGI.	CGI is more expensive than Servlets.
Servlets can handle the cookies.	CGI cannot handle the cookies.

Servlets API's:

Servlets are build from two packages:

- javax.servlet(Basic)
- javax.servlet.http(Advance)

Various classes and interfaces present in these packages are:

Component	Type	Package
Servlet	Interface	javax.servlet.*

Component	Type	Package
ServletRequest	Interface	javax.servlet.*
ServletResponse	Interface	javax.servlet.*
GenericServlet	Class	javax.servlet.*
HttpServlet	Class	javax.servlet.http.*
HttpServletRequest	Interface	javax.servlet.http.*
HttpServletResponse	Interface	javax.servlet.http.*
Filter	Interface	javax.servlet.*
ServletConfig	Interface	javax.servlet.*

Advantages of a Java Servlet

- Servlet is **faster** than CGI as it doesn't involve the creation of a new process for every new request received.
- Servlets, as written in Java, are platform-independent.
- Removes the overhead of creating a **new process** for each request as Servlet doesn't run in a separate process. There is only a single instance that handles all requests concurrently. This also saves the memory and allows a Servlet to easily manage the client state.
- It is a server-side component, so Servlet inherits the **security** provided by the Web server.
- The **API** designed for Java Servlet automatically acquires the advantages of the Java platforms such as platform-independent and portability. In addition, it obviously can use the wide range of APIs created on Java platforms such as **JDBC** to access the database.
- Many Web servers that are suitable for personal use or low-traffic websites are offered for free or at extremely **cheap costs** eg. Java servlet. However, the majority of commercial-grade Web servers are rather expensive, with the notable exception of Apache, which is free.

The Servlet Container

Servlet container, also known as **Servlet engine** is an integrated set of objects that provide a run time environment for Java Servlet components.

In simple words, it is a system that manages Java Servlet components on top of the Web server to handle the Web client requests.

Services provided by the Servlet container :

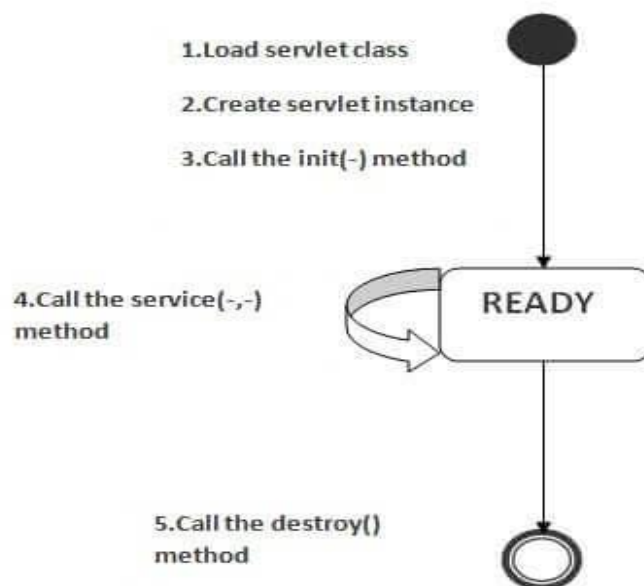
- **Network Services:** Loads a Servlet class. The loading may be from a local file system, a remote file system or other network services. The Servlet container provides the network services over which the request and response are sent.

- **Decode and Encode MIME-based messages:** Provides the service of decoding and encoding MIME-based messages.
- **Manage Servlet container:** Manages the lifecycle of a Servlet.
- **Resource management** Manages the static and dynamic resources, such as HTML files, Servlets, and JSP pages.
- **Security Service:** Handles authorization and authentication of resource access.
- **Session Management:** Maintains a session by appending a **session ID** to the URL path.

Life Cycle of a Servlet (Servlet Life Cycle)

The web container maintains the life cycle of a servlet instance. Let's see the life cycle of the servlet:

1. Servlet class is loaded.
2. Servlet instance is created.
3. init method is invoked.
4. service method is invoked.
5. destroy method is invoked.



As displayed in the above diagram, there are three states of a servlet: new, ready and end. The servlet is in new state if servlet instance is created. After invoking the `init()` method, Servlet comes in the ready state. In the ready state, servlet performs all the tasks. When the web container invokes the `destroy()` method, it shifts to the end state.

1) Servlet class is loaded

The classloader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

2) Servlet instance is created

The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

3) init method is invoked

The web container calls the `init` method only once after creating the servlet instance. The `init` method is used to initialize the servlet. It is the life cycle method of the `javax.servlet.Servlet` interface. Syntax of the `init` method is given below:

1. `public void init(ServletConfig config) throws ServletException`

4) service method is invoked

The web container calls the `service` method each time when request for the servlet is received. If servlet is not initialized, it follows the first three steps as described above then calls the `service` method. If servlet is initialized, it calls the `service` method. Notice that servlet is initialized only once. The syntax of the `service` method of the `Servlet` interface is given below:

1. `public void service(ServletRequest request, ServletResponse response)`
2. `throws ServletException, IOException`

5) destroy method is invoked

The web container calls the `destroy` method before removing the servlet instance from the service. It gives the servlet an opportunity to clean up any resource for example memory, thread etc. The syntax of the `destroy` method of the `Servlet` interface is given below:

1. `public void destroy()`
-

javax.servlet

Packages that use javax.servlet	
javax.servlet	The javax.servlet package contains a number of classes and interfaces that describe and define the contracts between a servlet class and the runtime environment provided for an instance of such a class by a conforming servlet container.
javax.servlet.http	The javax.servlet.http package contains a number of classes and interfaces that describe and define the contracts between a servlet class running under the HTTP protocol and the runtime environment provided for an instance of such a class by a conforming servlet container.

Classes in javax.servlet used by javax.servlet	
FilterChain	A FilterChain is an object provided by the servlet container to the developer giving a view into the invocation chain of a filtered request for a resource.
FilterConfig	A filter configuration object used by a servlet container to pass information to a filter during initialization.
RequestDispatcher	Defines an object that receives requests from the client and sends them to any resource (such as a servlet, HTML file, or JSP file) on the server.
Servlet	Defines methods that all servlets must implement.
ServletConfig	A servlet configuration object used by a servlet container to pass information to a servlet during initialization.
ServletContext	Defines a set of methods that a servlet uses to communicate with its servlet container, for example, to get the MIME type of a file, dispatch requests, or write to a log file.
ServletContextAttributeEvent	This is the event class for notifications about changes to the attributes of the servlet context of a web application.
ServletContextEvent	This is the event class for notifications about changes to the servlet context of a web application.
ServletException	Defines a general exception a servlet can throw when it encounters difficulty.
ServletInputStream	Provides an input stream for reading binary data from a client request, including an efficient readLine method for reading data one line at a time.
ServletOutputStream	Provides an output stream for sending binary data to the client.
ServletRequest	Defines an object to provide client request information to a servlet.
ServletRequestAttributeEvent	This is the event class for notifications of changes to the attributes of the servlet request in an application.
ServletRequestEvent	

Events of this kind indicate lifecycle events for a <code>ServletRequest</code> .	
<u>ServletResponse</u> Defines an object to assist a servlet in sending a response to the client.	

Classes in javax.servlet used by javax.servlet.http	
<u>GenericServlet</u> Defines a generic, protocol-independent servlet.	
<u>Servlet</u> Defines methods that all servlets must implement.	
<u>ServletConfig</u> A servlet configuration object used by a servlet container to pass information to a servlet during initialization.	
<u>ServletContext</u> Defines a set of methods that a servlet uses to communicate with its servlet container, for example, to get the MIME type of a file, dispatch requests, or write to a log file.	
<u>ServletException</u> Defines a general exception a servlet can throw when it encounters difficulty.	
<u>ServletInputStream</u> Provides an input stream for reading binary data from a client request, including an efficient <code>readLine</code> method for reading data one line at a time.	
<u>ServletRequest</u> Defines an object to provide client request information to a servlet.	
<u>ServletRequestWrapper</u> Provides a convenient implementation of the <code>ServletRequest</code> interface that can be subclassed by developers wishing to adapt the request to a Servlet.	
<u>ServletResponse</u> Defines an object to assist a servlet in sending a response to the client.	
<u>ServletResponseWrapper</u> Provides a convenient implementation of the <code>ServletResponse</code> interface that can be subclassed by developers wishing to adapt the response from a Servlet.	

Reading Servlet Parameters

The **ServletRequest** interface includes methods that allow you to read the names and values of parameters that are included in a client request. We will develop a servlet that illustrates their use. The example contains two files. A web page is defined in **PostParameters.html**, and a servlet is defined in **PostParametersServlet.java**.

The HTML source code for **PostParameters.html** is shown in the following listing. It defines a table that contains two labels and two text fields. One of the labels is Employee and the other is Phone. There is also a submit button. Notice that the action parameter of the form tag specifies a URL. The URL identifies the servlet to process the HTTP POST request.

```
<html>

<body>

<center>

<form name="Form1" method="post"
action="http://localhost:8080/examples/servlets/
servlet/PostParametersServlet">

<table>

<tr>

<td><B>Employee</td>

<td><input type="text" name="e" size="25" value=""></td> </tr>

<tr>

<td><B>Phone</td>

<td><input type="text" name="p" size="25" value=""></td> </tr>

</table>

<input type="submit" value="Submit"> </body>

</html>
```

The source code for **PostParametersServlet.java** is shown in the following listing. The **service()** method is overridden to process client requests. The **getParameterNames()** method returns an enumeration of the parameter names. These are processed in a loop. You can see that the parameter name and value are output to the client. The parameter value is obtained via the **getParameter()** method.

```
import java.io.*; import java.util.*; import javax.servlet.*;

public class PostParametersServlet extends GenericServlet {
```



```

public void service(ServletRequest request, ServletResponse response)
throws ServletException, IOException {
    // Get print writer.
    PrintWriter pw = response.getWriter();

    Get enumeration of parameter names. Enumeration e = request.getParameterNames();

    Display parameter names and values. while(e.hasMoreElements()) {
String pname = (String)e.nextElement(); pw.print(pname + " = ");
String pvalue = request.getParameter(pname); pw.println(pvalue);
    }

    pw.close();
}

```

Compile the servlet. Next, copy it to the appropriate directory, and update the **web.xml** file, as previously described. Then, perform these steps to test this example:

Start Tomcat (if it is not already running).

Display the web page in a browser.

Enter an employee name and phone number in the text fields.

Submit the web page.

After following these steps, the browser will display a response that is dynamically generated by the servlet.

The **javax.servlet.http** Package

The preceding examples have used the classes and interfaces defined in **javax.servlet**, such as **ServletRequest**, **ServletResponse**, and **GenericServlet**, to illustrate the basic functionality of servlets. However, when working with HTTP, you will normally use the interfaces and classes in **javax.servlet.http**. As you will see, its functionality makes it easy to build servlets that work with HTTP requests and responses.

The following table summarizes the interfaces used in this chapter:

Interface	Description
HttpServletRequest	Enables servlets to read data from an HTTP request.
HttpServletResponse	Enables servlets to write data to an HTTP response.
HttpSession	Allows session data to be read and written.

Interface : Description

HttpServletRequest : Enables servlets to read data from an HTTP request.

HttpServletResponse : Enables servlets to write data to an HTTP response.

HttpSession : Allows session data to be read and written.

The following table summarizes the classes used in this chapter. The most important of these is **HttpServlet**. Servlet developers typically extend this class in order to process HTTP requests.

Class	Description
Cookie	Allows state information to be stored on a client machine.
HttpServlet	Provides methods to handle HTTP requests and responses.

Class : Description

Cookie : Allows state information to be stored on a client machine.

HttpServlet : Provides methods to handle HTTP requests and responses.

The HttpServletRequest Interface

The **HttpServletRequest** interface enables a servlet to obtain information about a client request. Several of its methods are shown in Table 38-5.

The HttpServletResponse Interface

The **HttpServletResponse** interface enables a servlet to formulate an HTTP response to a client. Several constants are defined. These correspond to the different status codes that can be assigned to an HTTP response. For example, **SC_OK** indicates that the HTTP

Method	Description
String getAuthType()	Returns authentication scheme.
Cookie[] getCookies()	Returns an array of the cookies in this request.
long getDateHeader(String <i>field</i>)	Returns the value of the date header field named <i>field</i> .
String getHeader(String <i>field</i>)	Returns the value of the header field named <i>field</i> .
Enumeration<String> getHeaderNames()	Returns an enumeration of the header names.
int getIntHeader(String <i>field</i>)	Returns the int equivalent of the header field named <i>field</i> .
String getMethod()	Returns the HTTP method for this request.
String getPathInfo()	Returns any path information that is located after the servlet path and before a query string of the URL.
String getPathTranslated()	Returns any path information that is located after the servlet path and before a query string of the URL after translating it to a real path.
String getQueryString()	Returns any query string in the URL.
String getRemoteUser()	Returns the name of the user who issued this request.
String getRequestedSessionId()	Returns the ID of the session.
String getRequestURI()	Returns the URL.
StringBuffer getRequestURL()	Returns the URL.
String getServletPath()	Returns that part of the URL that identifies the servlet.

HttpSession getSession()	Returns the session for this request. If a session does not exist, one is created and then returned.
HttpSession getSession(boolean <i>new</i>)	If <i>new</i> is true and no session exists, creates and returns a session for this request. Otherwise, returns the existing session for this request.
boolean isRequestedSessionIdFromCookie()	Returns true if a cookie contains the session ID. Otherwise, returns false .
boolean isRequestedSessionIdFromURL()	Returns true if the URL contains the session ID. Otherwise, returns false .
boolean isRequestedSessionIdValid()	Returns true if the requested session ID is valid in the current session context.

Table 38-5 Various Methods Defined by **HttpServletRequest**

request succeeded, and **SC_NOT_FOUND** indicates that the requested resource is not available. Several methods of this interface are summarized in Table 38-6.

The HttpSession Interface

The **HttpSession** interface enables a servlet to read and write the state information that is associated with an HTTP session. Several of its methods are summarized in Table 38-7. All of these methods throw an **IllegalStateException** if the session has already been invalidated.

Method	Description
<code>void addCookie(Cookie <i>cookie</i>)</code>	Adds <i>cookie</i> to the HTTP response.
<code>boolean containsHeader(String <i>field</i>)</code>	Returns true if the HTTP response header contains a field named <i>field</i> .
<code>String encodeURL(String <i>url</i>)</code>	Determines if the session ID must be encoded in the URL identified as <i>url</i> . If so, returns the modified version of <i>url</i> . Otherwise, returns <i>url</i> . All URLs generated by a servlet should be processed by this method.
<code>String encodeRedirectURL(String <i>url</i>)</code>	Determines if the session ID must be encoded in the URL identified as <i>url</i> . If so, returns the modified version of <i>url</i> . Otherwise, returns <i>url</i> . All URLs passed to <code>sendRedirect()</code> should be processed by this method.
<code>void sendError(int <i>c</i>)</code> throws <code>IOException</code>	Sends the error code <i>c</i> to the client.
<code>void sendError(int <i>c</i>, String <i>s</i>)</code> throws <code>IOException</code>	Sends the error code <i>c</i> and message <i>s</i> to the client.
<code>void sendRedirect(String <i>url</i>)</code> throws <code>IOException</code>	Redirects the client to <i>url</i> .
<code>void setDateHeader(String <i>field</i>, long <i>msec</i>)</code>	Adds <i>field</i> to the header with date value equal to <i>msec</i> (milliseconds since midnight, January 1, 1970, GMT).
<code>void setHeader(String <i>field</i>, String <i>value</i>)</code>	Adds <i>field</i> to the header with value equal to <i>value</i> .
<code>void setIntHeader(String <i>field</i>, int <i>value</i>)</code>	Adds <i>field</i> to the header with value equal to <i>value</i> .
<code>void setStatus(int <i>code</i>)</code>	Sets the status code for this response to <i>code</i> .

Table 38-6 Various Methods Defined by **HttpServletResponse**

The Cookie Class

The **Cookie** class encapsulates a cookie. A *cookie* is stored on a client and contains state information. Cookies are valuable for tracking user activities. For example, assume that a user visits an online store. A cookie can save the user's name, address, and other information. The user does not need to enter this data each time he or she visits the store.

A servlet can write a cookie to a user's machine via the `addCookie()` method of the **HttpServletResponse** interface. The data for that cookie is then included in the header of the HTTP response that is sent to the browser.

The names and values of cookies are stored on the user's machine. Some of the information that can be saved for each cookie includes the following:

The name of the cookie

The value of the cookie

The expiration date of the cookie

The domain and path of the cookie

Method	Description
Object getAttribute(String <i>attr</i>)	Returns the value associated with the name passed in <i>attr</i> . Returns null if <i>attr</i> is not found.
Enumeration<String> getAttributeNames()	Returns an enumeration of the attribute names associated with the session.
long getCreationTime()	Returns the creation time (in milliseconds since midnight, January 1, 1970, GMT) of the invoking session.
String getId()	Returns the session ID.
long getLastAccessedTime()	Returns the time (in milliseconds since midnight, January 1, 1970, GMT) when the client last made a request on the invoking session.
void invalidate()	Invalidates this session and removes it from the context.
boolean isNew()	Returns true if the server created the session and it has not yet been accessed by the client.
void removeAttribute(String <i>attr</i>)	Removes the attribute specified by <i>attr</i> from the session.
void setAttribute(String <i>attr</i> , Object <i>val</i>)	Associates the value passed in <i>val</i> with the attribute name passed in <i>attr</i> .

Table 38-7 Various Methods Defined by **HttpSession**

The expiration date determines when this cookie is deleted from the user's machine. If an expiration date is not explicitly assigned to a cookie, it is deleted when the current browser session ends.

The domain and path of the cookie determine when it is included in the header of an HTTP request. If the user enters a URL whose domain and path match these values, the cookie is then supplied to the web server. Otherwise, it is not.

There is one constructor for **Cookie**. It has the signature shown here: `Cookie(String name, String value)`

Here, the name and value of the cookie are supplied as arguments to the constructor. The methods of the **Cookie** class are summarized in Table 38-8.

The **HttpServlet** Class

The **HttpServlet** class extends **GenericServlet**. It is commonly used when developing servlets that receive and process HTTP requests. The methods defined by the **HttpServlet** class are summarized in Table 38-9.

Method : Description

Object clone() : Returns a copy of this object.

String getComment() : Returns the comment.

String getDomain() : Returns the domain.

`int getMaxAge( )` : Returns the maximum age (in seconds).

`String getName( )` : Returns the name.

`String getPath( )` : Returns the path.

`boolean getSecure( )` : Returns true if the cookie is secure. Otherwise, returns false.

`String getValue( )` : Returns the value.

`int getVersion( )` : Returns the version.

`boolean isHttpOnly(?)` : Returns true if the cookie has the `HttpOnly` attribute.

`void setComment(String c)` : Sets the comment to *c*.

`void setDomain(String d)` : Sets the domain to *d*.

`void setHttpOnly(boolean httpOnly)` : If `httpOnly` is true, then the `HttpOnly` attribute is added to the cookie. If `httpOnly` is false, the `HttpOnly` attribute is removed.

`void setMaxAge(int secs)` : Sets the maximum age of the cookie to *secs*. This is the number of seconds after which the cookie is deleted.

`void setPath(String p)` : Sets the path to *p*.

`void setSecure(boolean secure)` : Sets the security flag to *secure*.

`void setValue(String v)` : Sets the value to *v*.

`void setVersion(int v)` : Sets the version to *v*.

Table 38-8 The Methods Defined by Cookie

Method	Description
<code>Object clone()</code>	Returns a copy of this object.
<code>String getComment()</code>	Returns the comment.
<code>String getDomain()</code>	Returns the domain.
<code>int getMaxAge()</code>	Returns the maximum age (in seconds).
<code>String getName()</code>	Returns the name.
<code>String getPath()</code>	Returns the path.
<code>boolean getSecure()</code>	Returns true if the cookie is secure. Otherwise, returns false .
<code>String getValue()</code>	Returns the value.
<code>int getVersion()</code>	Returns the version.
<code>boolean isHttpOnly()</code>	Returns true if the cookie has the HttpOnly attribute.
<code>void setComment(String c)</code>	Sets the comment to <i>c</i> .
<code>void setDomain(String d)</code>	Sets the domain to <i>d</i> .
<code>void setHttpOnly(boolean httpOnly)</code>	If <i>httpOnly</i> is true , then the HttpOnly attribute is added to the cookie. If <i>httpOnly</i> is false , the HttpOnly attribute is removed.
<code>void setMaxAge(int secs)</code>	Sets the maximum age of the cookie to <i>secs</i> . This is the number of seconds after which the cookie is deleted.
<code>void setPath(String p)</code>	Sets the path to <i>p</i> .
<code>void setSecure(boolean secure)</code>	Sets the security flag to <i>secure</i> .
<code>void setValue(String v)</code>	Sets the value to <i>v</i> .
<code>void setVersion(int v)</code>	Sets the version to <i>v</i> .

Table 38-8 The Methods Defined by **Cookie**

Method	Description
void doDelete(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP DELETE request.
void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP GET request.
void doHead(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP HEAD request.
void doOptions(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP OPTIONS request.

Table 38-9 The Methods Defined by **HttpServlet**

Method	Description
void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP POST request.
void doPut(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP PUT request.
void doTrace(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP TRACE request.
long getLastModified(HttpServletRequest req)	Returns the time (in milliseconds since midnight, January 1, 1970, GMT) when the requested resource was last modified.
void service(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Called by the server when an HTTP request arrives for this servlet. The arguments provide access to the HTTP request and response, respectively.

Table 38-9 The Methods Defined by **HttpServlet** (continued)

Handling HTTP Requests and Responses

The **HttpServlet** class provides specialized methods that handle the various types of HTTP requests. A servlet developer typically overrides one of these methods. These methods are **doDelete()**, **doGet()**, **doHead()**, **doOptions()**, **doPost()**, **doPut()**, and **doTrace()**. A complete description of the different types of HTTP requests is beyond the scope of this book. However, the GET and POST requests are commonly used when handling form input. Therefore, this section presents examples of these cases.

Handling HTTP GET Requests

Here we will develop a servlet that handles an HTTP GET request. The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in **ColorGet.html**, and a servlet is defined in **ColorGetServlet.java**. The HTML source code for **ColorGet.html** is shown in the following listing. It defines a form that contains a select element and a submit button. Notice that the action parameter of the form tag specifies a URL. The URL identifies a servlet to process the HTTP GET request.

```
<html>

<body>

<center>

<form
name="Form1" action="http://localhost:8080/examples/servlets/servlet/ColorGetServlet">

<B>Color:</B>

<select    name="color"    size="1">    <option    value="Red">Red</option>    <option
value="Green">Green</option>

<option value="Blue">Blue</option> </select>

<br><br>

<input type="submit" value="Submit"> </form>

</body>

</html>
```

The source code for **ColorGetServlet.java** is shown in the following listing. The **doGet()** method is overridden to process any HTTP GET requests that are sent to this servlet. It uses the **getParameter()** method of **HttpServletRequest** to obtain the selection that was made by the user. A response is then formulated.

```
import java.io.*; import javax.servlet.*;

import javax.servlet.http.*;

public class ColorGetServlet extends HttpServlet {

public void doGet(HttpServletRequest request, HttpServletResponse response)

throws ServletException, IOException {

String color = request.getParameter("color"); response.setContentType("text/html"); PrintWriter pw
= response.getWriter(); pw.println("<B>The selected color is: "); pw.println(color);

pw.close();

}

}
```


Compile the servlet. Next, copy it to the appropriate directory, and update the **web.xml** file, as previously described. Then, perform these steps to test this example:

Start Tomcat, if it is not already running.

Display the web page in a browser.

Select a color.

Submit the web page.

After completing these steps, the browser will display the response that is dynamically generated by the servlet.

One other point: Parameters for an HTTP GET request are included as part of the URL that is sent to the web server. Assume that the user selects the red option and submits the form. The URL sent from the browser to the server is

`http://localhost:8080/examples/servlets/servlet/ColorGetServlet?color=Red`

The characters to the right of the question mark are known as the *query string*.

Handling HTTP POST Requests

Here we will develop a servlet that handles an HTTP POST request. The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in **ColorPost.html**, and a servlet is defined in **ColorPostServlet.java**.

The HTML source code for **ColorPost.html** is shown in the following listing. It is identical to **ColorGet.html** except that the method parameter for the form tag explicitly specifies that the POST method should be used, and the action parameter for the form tag specifies a different servlet.

```
<html>
```

```
<body>
```

```
<center>
```

```
<form name="Form1" method="post"
```

```
action="http://localhost:8080/examples/servlets/servlet/ColorPostServlet">
```

```
<B>Color:</B>
```

```

<select    name="color"    size="1">    <option    value="Red">Red</option>    <option
value="Green">Green</option> <option value="Blue">Blue</option> </select>
<br><br>

<input type=submit value="Submit"> </form>

</body>

</html>

```

The source code for **ColorPostServlet.java** is shown in the following listing. The **doPost()** method is overridden to process any HTTP POST requests that are sent to this servlet. It uses the **getParameter()** method of **HttpServletRequest** to obtain the selection that was made by the user. A response is then formulated.

```

import java.io.*; import javax.servlet.*;
import javax.servlet.http.*;

public class ColorPostServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)

        throws ServletException, IOException {

        String color = request.getParameter("color"); response.setContentType("text/html"); PrintWriter pw
        = response.getWriter(); pw.println("<B>The selected color is: "); pw.println(color);

        pw.close();

    }

}

```

Compile the servlet and perform the same steps as described in the previous section to test it.

Using Cookies

Now, let's develop a servlet that illustrates how to use cookies. The servlet is invoked when a form on a web page is submitted. The example contains three files as summarized here:

File : Description

AddCookie.html : Allows a user to specify a value for the cookie named MyCookie.

AddCookieServlet.java : Processes the submission of AddCookie.html.

GetCookiesServlet.java : Displays cookie values.

The HTML source code for **AddCookie.html** is shown in the following listing. This page contains a text field in which a value can be entered. There is also a submit button on the page. When this button is pressed, the value in the text field is sent to **AddCookieServlet** via an HTTP POST request.

```

<html>

<body>

<center>

<form name="Form1" method="post"

action="http://localhost:8080/examples/servlets/servlet/AddCookieServlet"> <B>Enter a value for
MyCookie:</B>

<input type="text" name="data" size=25 value=""> <input type="submit" value="Submit">

</form>

</body>

</html>;

```

The source code for **AddCookieServlet.java** is shown in the following listing. It gets the value of the parameter named "data". It then creates a **Cookie** object that has the name "MyCookie" and contains the value of the "data" parameter. The cookie is then added to the header of the HTTP response via the **addCookie()** method. A feedback message is then written to the browser.

```

import java.io.*; import javax.servlet.*;

import javax.servlet.http.*;

public class AddCookieServlet extends HttpServlet {

public void doPost(HttpServletRequest request, HttpServletResponse response)

throws ServletException, IOException {

// Get parameter from HTTP request.

String data = request.getParameter("data");

// Create cookie.

Cookie cookie = new Cookie("MyCookie", data);

    Add cookie to HTTP response. response.addCookie(cookie);

    Write output to browser. response.setContentType("text/html"); PrintWriter pw =
response.getWriter(); pw.println("<B>MyCookie has been set to"); pw.println(data);
pw.close();

}

}

```

The source code for **GetCookiesServlet.java** is shown in the following listing. It invokes the **getCookies()** method to read any cookies that are included in the HTTP GET request. The names and values of these cookies are then written to the HTTP response. Observe that the **getName()** and **getValue()** methods are called to obtain this information.

```

import java.io.*; import javax.servlet.*;

import javax.servlet.http.*;

```

```

public class GetCookiesServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        Get cookies from header of HTTP request. Cookie[] cookies = request.getCookies();
        Display these cookies. response.setContentType("text/html"); PrintWriter pw =
        response.getWriter(); pw.println("<B>");
        for(int i = 0; i < cookies.length; i++) { String name = cookies[i].getName(); String value =
        cookies[i].getValue(); pw.println("name = " + name +
        "; value = " + value);
        }
        pw.close();
    }
}

```

Compile the servlets. Next, copy them to the appropriate directory, and update the **web.xml** file, as previously described. Then, perform these steps to test this example:

Start Tomcat, if it is not already running.

Display **AddCookie.html** in a browser.

Enter a value for **MyCookie**.

Submit the web page.

After completing these steps, you will observe that a feedback message is displayed by the browser. Next, request the following URL via the browser:

<http://localhost:8080/examples/servlets/servlet/GetCookiesServlet>

Observe that the name and value of the cookie are displayed in the browser.

In this example, an expiration date is not explicitly assigned to the cookie via the **setMaxAge()** method of **Cookie**. Therefore, the cookie expires when the browser session ends. You can experiment by using **setMaxAge()** and observe that the cookie is then saved on the client machine.

Session Tracking

HTTP is a stateless protocol. Each request is independent of the previous one. However, in some applications, it is necessary to save state information so that information can be collected from several interactions between a browser and a server. Sessions provide such a mechanism.

A session can be created via the **getSession()** method of **HttpServletRequest**.

An **HttpSession** object is returned. This object can store a set of bindings that associate names with

objects. The **setAttribute()**, **getAttributeNames()**, and **removeAttribute()** methods), **getAttribute(of HttpSession** manage these bindings. Session state is shared by all servlets that are associated with a client.

The following servlet illustrates how to use session state. The **getSession()** method gets the current session. A new session is created if one does not already exist. The **getAttribute()** method is called to obtain the object that is bound to the name "date". That object is a **Date** object that encapsulates the date and time when this page was last accessed. (Of course, there is no such binding when the page is first accessed.) A **Date** object encapsulating the current date and time is then created. The **setAttribute()** method is called to bind the name "date" to this object.

```
import java.io.*; import java.util.*; import javax.servlet.*;
import javax.servlet.http.*;

public class DateServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //Get the HttpSession object.
        HttpSession hs = request.getSession(true);

        //Get writer.
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter(); pw.print("<B>");

        //Display date/time of last access.

        Date date = (Date)hs.getAttribute("date"); if(date != null) {
            pw.print("Last access: " + date + "<br>");
        }

        // Display current date/time.
        date = new Date(); hs.setAttribute("date", date);
        pw.println("Current date: " + date);
    }
}
```

When you first request this servlet, the browser displays one line with the current date and time information. On subsequent invocations, two lines are displayed. The first line shows the date and time when the servlet was last accessed. The second line shows the current date and time.